

EXP 5: Dynamic Programming using Policy Iteration and Value Iteration

Aim

To implement Dynamic Programming techniques in Reinforcement Learning using Policy Iteration and Value Iteration, and determine the optimal policy and value function for a given environment.

Algorithm – Policy Iteration

1. Initialize a random policy and value function.
2. Evaluate the policy using the Bellman equation.
3. Improve the policy by selecting the best action.
4. Repeat evaluation and improvement until the policy becomes stable.
5. Return the optimal policy and value function.

Algorithm – Value Iteration

1. Initialize the value function to zero.
2. Calculate the value of each possible action.
3. Select the maximum value for each state.
4. Repeat until the values converge.
5. Extract the optimal policy from the final values.

CODE

```
import numpy as np
import matplotlib.pyplot as plt

states = 3
actions = 2
gamma = 0.9
rewards = np.array([
    [0, 1],
    [0, 5],
    [10, 10]
```

```

])
P = np.zeros((states, actions, states))
P[0, 0, 0] = 1
P[0, 1, 1] = 1
P[1, 0, 0] = 1
P[1, 1, 2] = 1
P[2, 0, 2] = 1
P[2, 1, 2] = 1
V = np.zeros(states)
value_history = []
for iteration in range(50):
    new_V = np.zeros(states)
    for state in range(states):
        action_values = []
        for action in range(actions):

            value = (
rewards[state, action]
            + gamma * np.sum(
                P[state, action] * V
            )
        )
        action_values.append(value)
    new_V[state] = max(action_values)
    V = new_V.copy()
    value_history.append(V.copy())
policy = []
for state in range(states):
    action_values = []
    for action in range(actions):
        value = (

```

```

rewards[state, action]
+ gamma * np.sum( P[state, action] * V
)
)
action_values.append(value) best_action = np.argmax(action_values) policy.append(best_action)
print("=" * 55)
print(" DYNAMIC PROGRAMMING - VALUE ITERATION")
print("=" * 55) print("\nFinal State Values:") for state in range(states):
print(
"State", state,

".",
round(V[state], 2)
)
print("\nOptimal Policy:") for state in range(states):
if policy[state] == 0: action_name = "Left"
else:
action_name = "Right" print(
"State", state, "->",
action_name
)
print("\nNumber of Iterations:", len(value_history)) value_history = np.array(value_history)
plt.figure(figsize=(9, 5))
plt.plot(
value_history[:, 0],
label="State 0"
)
plt.plot(
value_history[:, 1],
label="State 1"
)

plt.plot(
value_history[:, 2],
label="State 2"
)
plt.xlabel("Iteration") plt.ylabel("State Value")
plt.title("Value Iteration Convergence") plt.legend()
plt.grid(True) plt.tight_layout() plt.show() plt.figure(figsize=(7, 5)) plt.bar(
["State 0", "State 1", "State 2"], V
)
plt.xlabel("States") plt.ylabel("Value") plt.title("Final State Values") plt.grid(axis="y") plt.tight_layout()
plt.show()

```

OUTPUT

DYNAMIC

PROGRAMMING - VALUE ITERATION

Final State Values:

State 0 : 85.98

State 1 : 94.48

State 2 : 99.48

Optimal Policy:

State 0 -> Right State 1 -> Right State 2 -> Le



